

University of M'hammed Bougara,
Boumerdes, Algeria

Department of Computer Science,
Faculty of Sciences,

Free and Open Source Software

Presented by: Aicha Benyoucef

2025/2026





Chapter 2: Open source tools

- 2.1. Introduction (history, advantages/disadvantages, and licensing)
- 2.2. Development environment (introduction to Linux, introduction to code editors)
- 2.3. Office automation (LibreOffice office suite)
- 2.4. Collaboration (storage and sharing)
- 2.5. Contributing to an open source project



2.1 Introduction

a. What is Free Software?

Free software is software distributed under a license that guarantees the user four essential freedoms:

1. Freedom to use the software for any purpose.
2. Freedom to study how the software works (access to the source code).
3. Freedom to modify it.
4. Freedom to redistribute copies, whether modified or not.

Free software is often confused with “free of charge,” but the term refers to freedom, not price.



**WHAT IS:
FREE SOFTWARE**

ThimPress





2.1 Introduction

b. You already use free software.

Directly:

- Your operating system if you're using GNU/Linux,
- Your web browser if it's Firefox or Chrome,
- Part of your phone if it runs Android,
- Your text editor if it's Emacs, Vim, VS Code, or Atom,
- The compiler for your preferred programming language,
- The Git version manager, etc.

Indirectly (through dependencies on free libraries):

- Most of the websites you browse,
- Most of the mobile applications you use.

2.1 Introduction

c. Why this course is useful for you?



As future engineers.



As future researchers.



On a personal level.

2.1 Introduction



As future engineers

- To get hired: your GitHub profile is your second résumé.
- Using free software is essential in almost all companies (inside and outside the IT sector):
 - Cost reduction,
 - Adaptability,
 - Sustainability.
 - Free software can also drive collaborations beyond the company's boundaries.

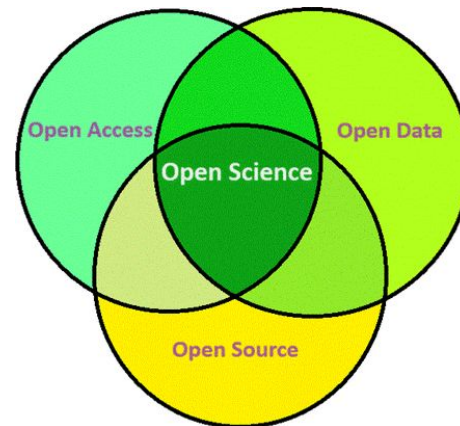


2.1 Introduction



As future researchers

- The three pillars of open science: **open access**, **open data**, **open source**.
- Sharing source code as open source → reproducing research results, building on existing work, comparing different techniques.
- An opportunity to create international research communities around projects (e.g., Coq, Scikit-learn, etc.).



2.1 Introduction



On a personal level

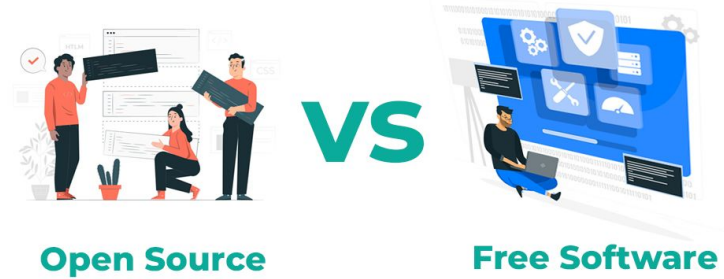
When you use free software, you can:

- report bugs,
- view and modify the source code to fix issues,
- adapt it to your needs,
- be part of a community.

If you develop software or a library, you can share it and receive contributions.



2.1 Introduction



d. Free software vs. open source

Open Source (OS I, 1998)

Focus on **collaboration and code sharing** (Open Source Definition).

Common confusion: *publishing source code* $\neq$ *open source* if the license is restrictive.

Free Software (S tallman, FS F, 1985)

Focus on **user freedoms** (Free Software Definition): 0. Use for any purpose

1. Study & modify (requires source code)
 2. Redistribute copies
 3. Redistribute modified versions
- Ambiguity of “free”

FOSS / FLOSS

Neutral terms combining Free / Libre / Open Source Software.

In practice, almost all free software is open source, and vice versa.



2.1 Introduction



e. Community, Transparency, and Code Sharing in Open Source

Community

- Developed by a global community of users and developers
- Collaboration through bug reports, feature requests, and contributions
- Encourages innovation and continuous improvement

Transparency

- Source code is publicly available
- Users can inspect, audit, and understand how the software works
- Improves trust, security, and software quality

Code Sharing

- Source code can be used, modified, and redistributed
- Shared under open source **licenses**
- Promotes knowledge sharing and prevents vendor lock-in



2.1 Introduction

f. Open Source Licenses

1- Definition of an Open Source License

An open source license is a **legal agreement** that defines how software source code can be used, modified, and redistributed.

It guarantees users the freedom to:

- **Access** the source code
- **Use** the software for any purpose
- **Modify** and **improve** the software
- **Share** original or modified versions

Open source licenses protect both the developers' rights and the users' freedoms.



2.1 Introduction

f. Open Source Licenses

2- Why Open Source Licenses Are Important

Open source licenses:

- Ensure **legal use** of open source software
- **Protect** intellectual property
- Define user and developer **responsibilities**
- **Promote** collaboration and innovation

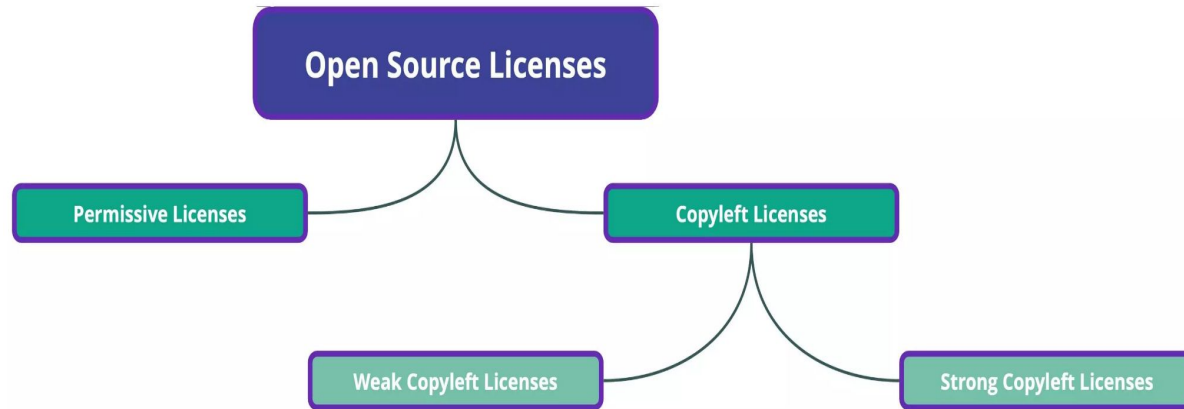
Without a license, software is automatically protected by copyright, even if the code is public.



2.1 Introduction

f. Open Source Licenses

3- Types of Open Source Licenses





2.1 Introduction

f. Open Source Licenses

3- Types of Open Source Licenses

3.1 Permissive Licenses

Permissive licenses allow users to use, modify, and redistribute the software with very few restrictions.

Key characteristics:

- Code can be used in proprietary (closed-source) software
- Minimal legal obligations
- High flexibility for companies



2.1 Introduction

f. Open Source Licenses

3- Types of Open Source Licenses

3.1 Permissive Licenses

Exemples :

- **MIT License:** The MIT License is known for its brevity and clarity. It grants permission to use, modify, and distribute the software, with the condition that the original copyright notice and the license text are retained in the redistributed software.
- **BSD License:** The BSD (Berkeley Software Distribution) license is a permissive open-source license that allows developers to use, modify, and distribute software with minimal restrictions.
- **Apache License 2.0:** The Apache 2.0 license contains a patent grant that ensure that authors that contribute code to an Apache 2.0 licensed project cannot restrict others from using the contribution by claiming their use infringes on a patent they hold.



2.1 Introduction

f. Open Source Licenses

3- Types of Open Source Licenses

3.2 Copyleft Licenses

Copyleft licenses require that modified or derived software must remain open source under the same license.

Key characteristics:

- Guarantees that software stays free and open
- Prevents proprietary reuse without sharing changes
- Strong protection of community contributions



2.1 Introduction

f. Open Source Licenses

3- Types of Open Source Licenses

3.1 Copyleft Licenses

Exemples :

- **GPL (General Public License):** The GPL is a widely-used, strong "copyleft" free software license created by Richard Stallman and the Free Software Foundation (FSF) that ensures users can freely run, study, share, and modify software, requiring that any derivative works also be released under the GPL, thereby preventing the software from becoming proprietary and promoting collaboration and openness
- **LGPL (Lesser General Public License):** The LGPL is a free-software license, primarily for libraries, that offers a middle ground between strong copyleft (like the GPL) and permissive licenses (like MIT) by applying weaker copyleft to the library itself, not the whole application using it, allowing proprietary apps to link to LGPL libraries without opening their own code, but requiring any modifications to the LGPL library to be released under the LGPL, ensuring the library remains free.



2.2. Development environment (introduction to Linux, introduction to code editors)

A development environment is a set of **software tools** and systems used to create, test, and maintain applications. In open source ecosystems, Linux operating systems and open source editors/IDEs play a central role.



2.2.1 Introduction to Linux

What is Linux

Unix-like computer **operating system** assembled under the model of free and open-source software development and distribution.

- These operating systems share the **Linux kernel**.
 - Typically have the GNU utilities
- Comes in several “distributions” to serve different purposes.

2.2.1 Introduction to Linux

Examples of Linux Distributions

A Linux distribution (or “distro”) is a complete operating system built around the Linux kernel plus system tools, desktop environment, installer, and package manager.

Each distribution combines these components differently and follows its own philosophy (stability, latest features, ease of use, or high customizability), which makes it better suited to particular types of users and use cases.



2.2.1 Introduction to Linux

Examples of Linux Distributions

Role of the distributions shown

- Debian, Ubuntu, and Linux Mint are often recommended for beginners on desktops and laptops because they focus on stability and ease of use.
- Fedora, openSUSE, CentOS, and Red Hat are widely used for servers, cloud, and enterprise environments, with stronger emphasis on support and predictable release cycles.
- Arch Linux and Gentoo target advanced users who want a very customizable system, often using a “rolling release” model that keeps software very up to date but requires more



2.2.1 Introduction to Linux

who is Linux



- Linux is an O/S core originally written by Linus Torvalds. Now almost 10,000 developers including major technology companies like Intel and IBM.



- A set of programs written by Richard Stallman and others. They are the GNU utilities.



2.2.1 Introduction to Linux

where is Linux

- World Wide Web
 - 67% of the world's web-servers run Linux (2016)
- Research/High-Performance Compute
 - Google, Amazon, NSA, 100% of TOP500 Super-computers.
- Modern Smartphones and devices
 - The Android phone
 - Amazon Kindle
 - Smart TVs/Devices



2.2.1 Introduction to Linux

Linux Application

- Servers and cloud computing
- Supercomputers
- Embedded systems
- Mobile devices (Android)
- Development environments

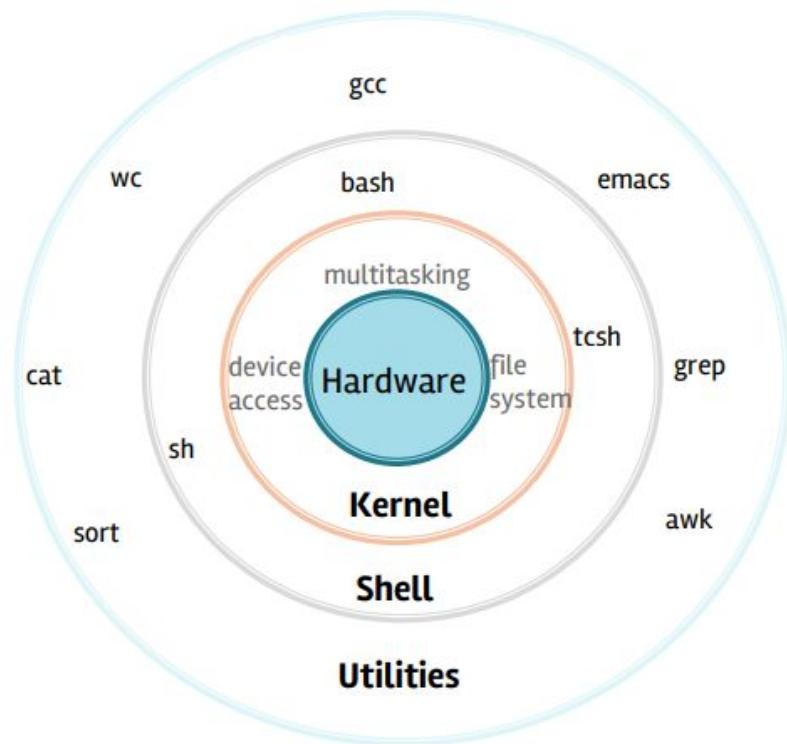


2.2.1 Introduction to Linux

Structure of the Linux System

Linux is composed of several layers:

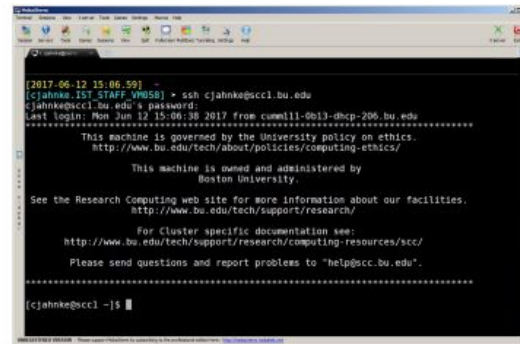
- **Kernel**
The core of the operating system; it manages hardware, memory, processes, and system resources.
- **Distribution (Distro)**
A complete operating system built around the Linux kernel, including system tools, libraries, and applications.
- **User Space**
Where applications, command-line tools, and graphical interfaces



2.2.1 Introduction to Linux

Linux Interaction / the Shell

- Program that *interprets commands* and sends them to the OS
- Provides:
 - Built-in commands
 - Programming control structures
 - Environment variables
- Linux supports multiple shells.
 - The default on SCC is **Bash**.



```
[2017-06-12 15:06:59] -  
[cjhahnke@sccl1.bu.edu] > ssh cjhahnke@sccl1.bu.edu  
cjhahnke@sccl1.bu.edu's password:  
Last login: Mon Jun 12 15:06:39 2017 from cuml11-0b13-dhcp-206.bu.edu  
=====
```

This machine is governed by the University policy on ethics.
<http://www.bu.edu/tech/about/policies/computing-ethics/>

This machine is owned and administered by
Boston University.

See the Research Computing web site for more information about our facilities.
<http://www.bu.edu/tech/support/research/>

For Cluster specific documentation see:
<http://www.bu.edu/tech/support/research/computing-resources/scc/>

Please send questions and report problems to "help@ecc.bu.edu".

```
=====
```

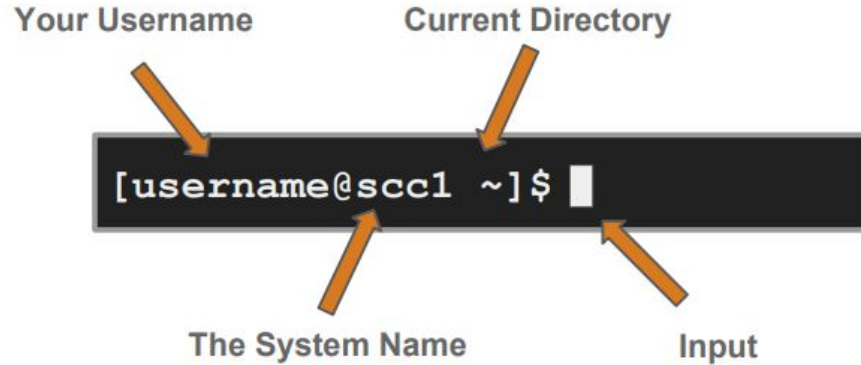
[cjhahnke@sccl1 ~]\$

"Bash" = "Bourne-again Shell"
(GNU version of ~1977 shell written by Stephen Bourne)



2.2.1 Introduction to Linux

Linux Interaction / The 'prompt'



(In Linux “ ~ ” is a shorthand for your home directory.)



2.2.1 Introduction to Linux

Basic Linux Usage/ commande

```
[username@scc1 ~]$ command --option argument
```

- **Command:** Command/program that does one thing
- **Options:** Change the way a command does that one thing
 - Short form: Single-dash and one letter e.g. **ls -a**
 - Long form: Double-dash and a word e.g. **ls --all**
- **Argument:** Provides the input/output that the command interacts with.

For more information about any command, use **man** or **info** (e.g. "**man ls**")



2.2.1 Introduction to Linux

Linux/ commande

Command	Description
pwd	Prints Current Working Directory
ls	Lists the contents of a directory
cd	Change the current path to the destination directory
mkdir	Makes a new directory
rmdir	Removes an empty directory
cp	Copy file or directory
mv	Move/Rename file or directory
rm	Delete file or directory
cat	Concatenates and prints the contents a file



2.2.1 Introduction to Linux

Linux/ commande

Command	Description
echo	Write arguments to the standard output
wc	word, line, character, and byte count
man	Search and open the manual page of a Linux command
more	Paging through text one screenful at a time
less	Improved version of more allows backward/forward movement
head	Display first lines of a file
tail	Display last lines of a file
grep	Print lines in a file matching a pattern
history	See the commands you have typed so far

2.2. 2 Introduction to code editors

An open source code editor is a software application designed for writing and editing source code, where the original source code is made freely available to the public. This allows anyone to view, copy, modify, and distribute the software's underlying code without restriction, subject to the terms of a specific open source license.



Sublime Text



Brackets



Visual Code
Editor



Bluefish



TextMate



TextWrangler

2.2. 2 Introduction to code editors

Classic Editors

Vs

Moderne Editors



Sublime Text



Brackets



Visual Code
Editor



Bluefish



TextMate



TextWrangler

2.2. 2 Introduction to code editors

Classic Editors

1- Vim / NeoVim

- Modal editor (command mode, insert mode)
- Extremely fast and lightweight
- Highly customizable
- Ideal for terminal-based development





2.2. 2 Introduction to code editors

Classic Editors

2- Emacs

- More than an editor: a complete computing environment
- Highly extensible using Emacs Lisp
- Strong philosophy: “Everything is customizable”



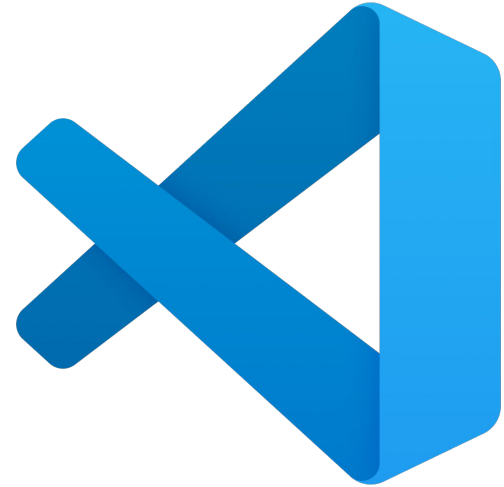


2.2. 2 Introduction to code editors

Modern Editors

1- VS Code (Open Source Base)

- Lightweight but powerful editor
- Supports many languages
- Large extension marketplace
- Integrated terminal and Git support





2.2. 2 Introduction to code editors

Modern Editors

2- Atom

- Hackable and customizable
- User-friendly interface
- Community-developed editor



2.2. 2 Introduction to code editors

Modern Editors

3- Geany

- Lightweight IDE-style editor
- Fast and simple
- Suitable for beginners



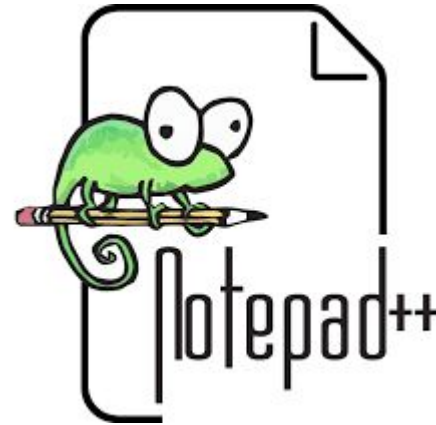
Geany

2.2. 2 Introduction to code editors

Modern Editors

4- Notepad++

- Windows-based open source editor
- Supports syntax highlighting
- Lightweight and easy to use





2.2.3 Integrated Development Environments (IDEs)

An IDE is a complete software suite for application development.

Examples of Open Source / Community IDEs

- **Eclipse** – Java, C/C++, web development
- **NetBeans** – Java, PHP, web applications
- **Code::Blocks** – C and C++ development
- **PyCharm Community Edition** – Python development



2.2.3 Integrated Development Environments (IDEs)

Essential Features of an IDE

- **Code autocompletion** – speeds up coding
- **Debugger** – finds and fixes errors
- **Project management** – organizes files and resources
- **Extensions / Plugins** – add extra functionality
- **Version control integration** – Git support





2.3. Office automation (LibreOffice office suite)

LibreOffice is a free and open-source office suite developed by *The Document Foundation*. It provides a complete set of applications for office automation, suitable for personal, academic, and professional use.



2.3. Office automation (LibreOffice office suite)

Free of Charge and Compatibility

- LibreOffice is **completely free** to download and use.
- It runs on Windows, Linux, and macOS.
- It is highly compatible with Microsoft Office formats (DOCX, XLSX, PPTX).
- Documents can be exported to PDF without additional software.

Main Components of the Suite

- **Writer** – Word processing
- **Calc** – Spreadsheet
- **Impress** – Presentation
- **Draw** – Diagrams and vector graphics
- **Base** – Database management
- **Math** – Equation editor



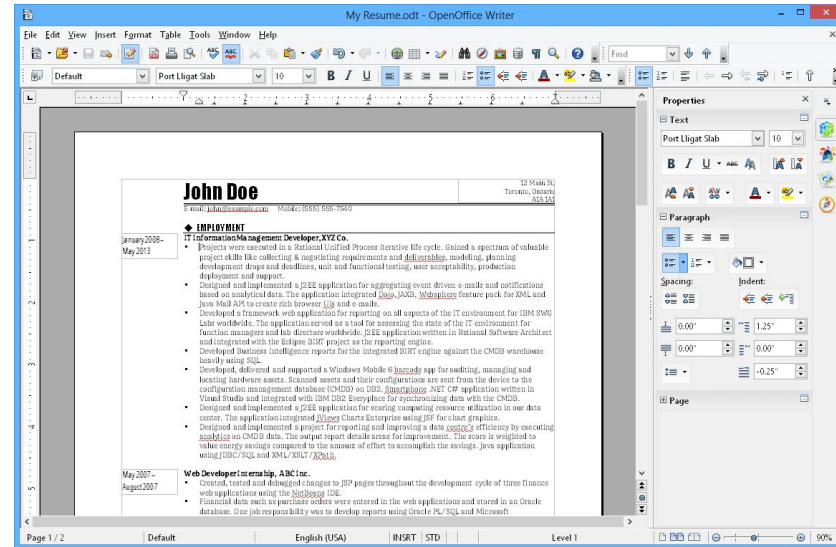
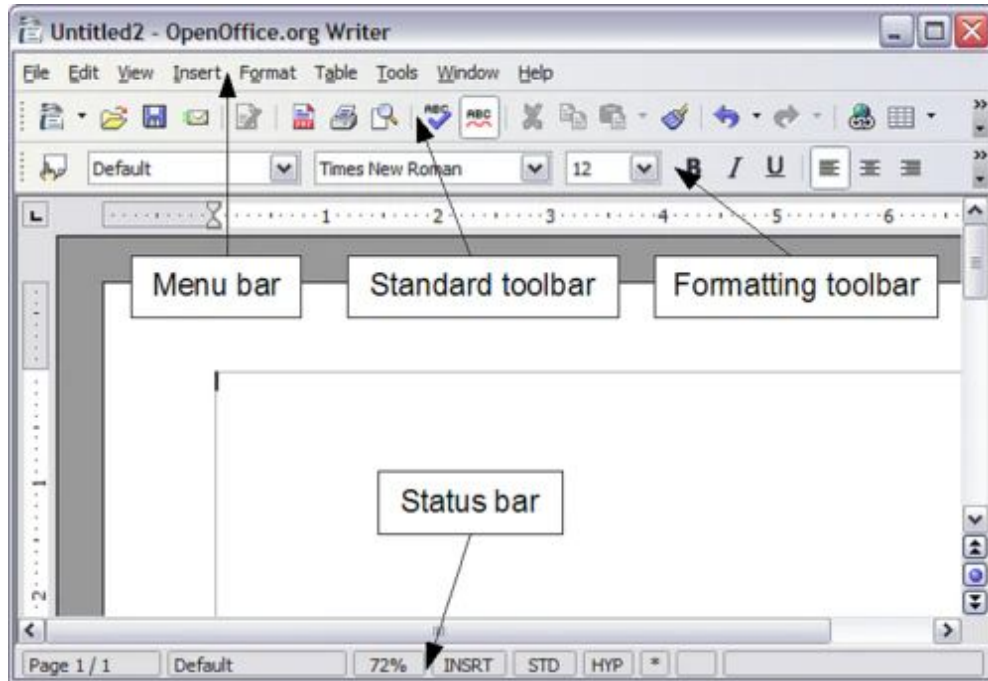
2.3.1 Writer (Word Processing)

LibreOffice Writer is used to create and edit text documents such as reports, letters, and academic documents.

Main Features

- Simple page layout: margins, page size, orientation, headers, and footers.
- Styles: paragraph styles, character styles, and page styles for consistent formatting.
- Sections: organize long documents.
- Tables and images: insertion and formatting of tables, images, and graphics.
- PDF export: documents can be exported directly to PDF format.

2.3.1 Writer (Word Processing)





2.3.2 Calc (Spreadsheet)

LibreOffice Calc is a spreadsheet application used for calculations, data analysis, and data visualization.

Basic Formulas: Mathematical and logical formulas such as SUM, AVERAGE, MIN, MAX, IF.

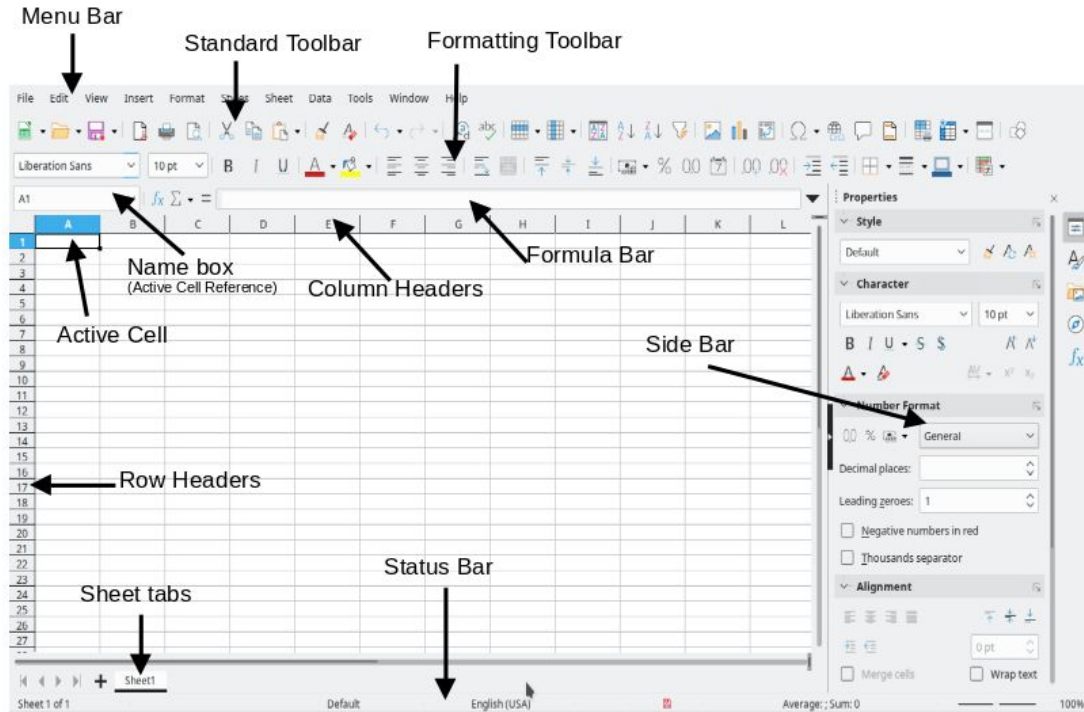
Data Management

- Sorting and filtering data.
- Pivot tables for data analysis and summaries.

Excel Compatibility

- Import and export Excel files (XLS, XLSX).
- Maintains formulas and formatting when exchanging files with Microsoft Excel.

2.3.3





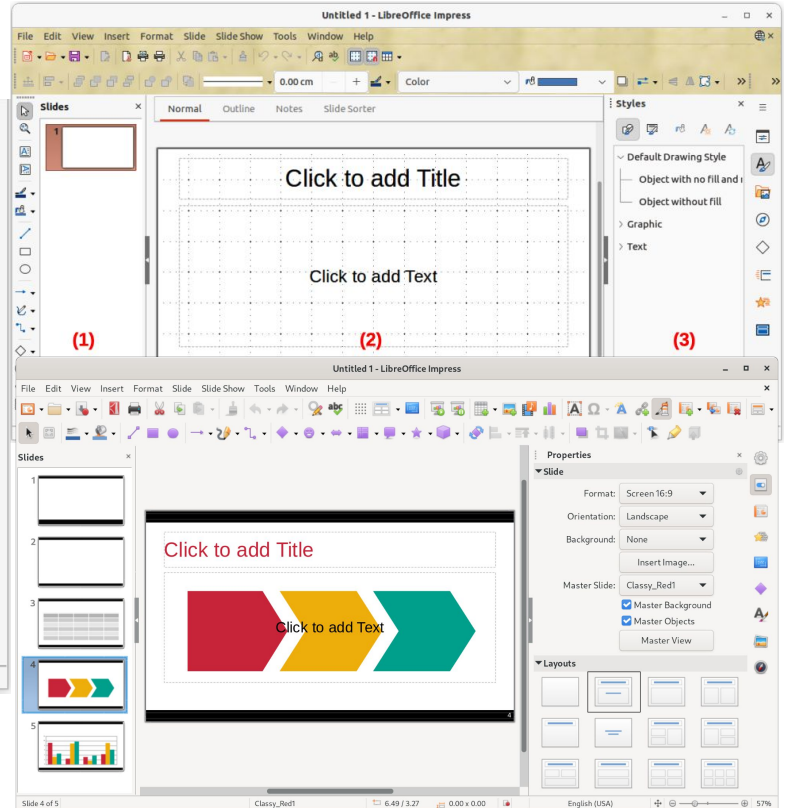
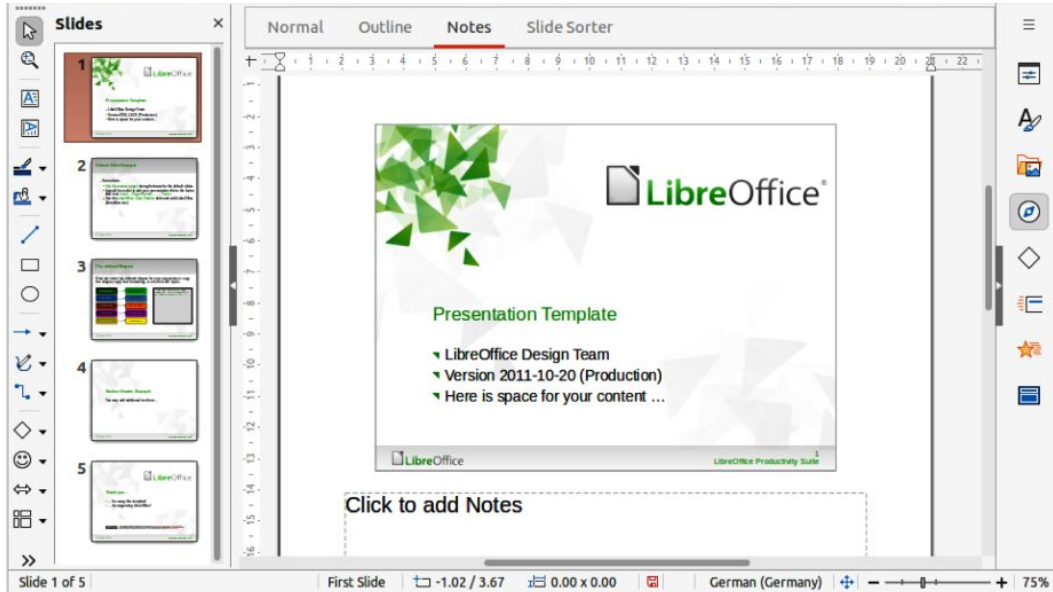
2.3.3 Impress (Presentation)

LibreOffice Impress is used to create multimedia presentations.

Main Features

- Slide creation with text, images, charts, and multimedia.
- Themes and templates for professional design.
- Transitions and animations between slides.
- Export options: presentations can be exported to PDF and PPTX formats.

2.3.3 Impress (Presentation)





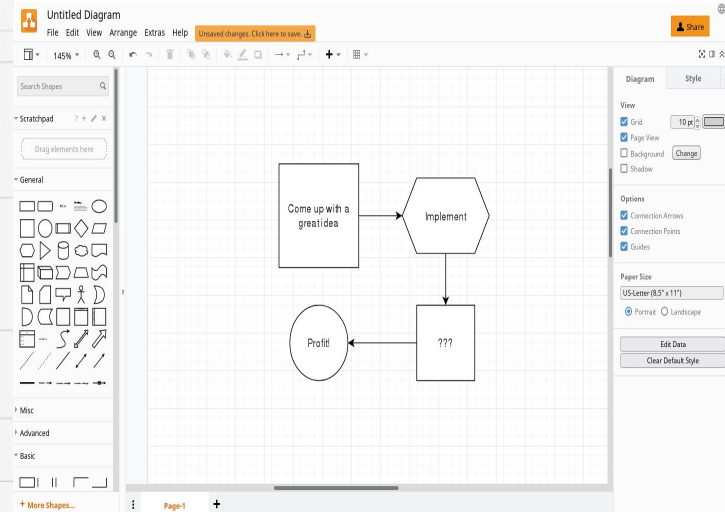
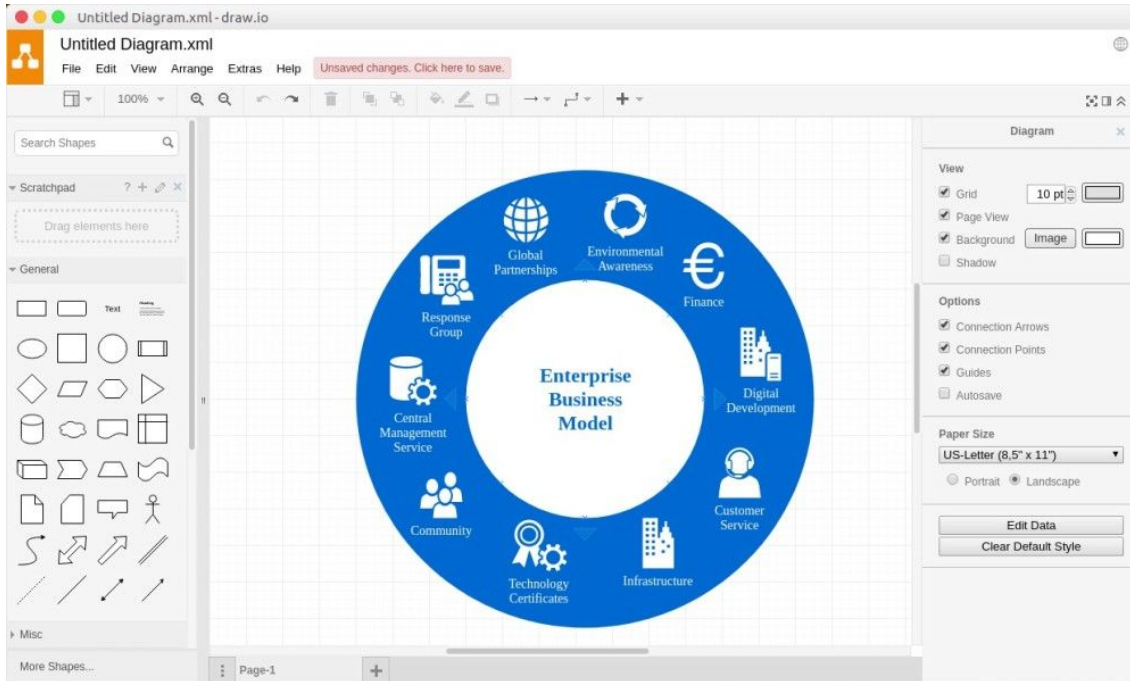
2.3.4 Draw

LibreOffice Draw is a tool for vector drawing and diagram creation.

Main Features

- Creation of diagrams, schemas, and flowcharts.
- Design of organizational charts.
- Simple technical drawings and illustrations.

2.3.4 Draw





2.3.4 Base

LibreOffice Base is a database management system designed for small to medium-scale databases.

Main Features

- Introduction to databases: tables, fields, and records.
- Forms for easy data entry.
- Reports for structured data output.
- Can connect to external databases (MySQL, PostgreSQL, etc.).



2.3.4 Math

LibreOffice Math is an equation editor used to write mathematical formulas.

Main Features

- Creation of mathematical equations (fractions, integrals, matrices).
- Integration with Writer, Calc, and Impress.
- Supports standard mathematical notation.

The screenshot shows the OpenOffice Math application window. The title bar reads "Answer To Universe.odf - OpenOffice Math". The menu bar includes "File", "Edit", "View", "Format", "Tools", "Window", and "Help". The toolbar contains various icons for file operations, editing, and mathematical functions. The main workspace displays the formula:

$$f(x, y) = \frac{e^M \cdot \sqrt{\cos(x) + \tan(y)}}{M^2}$$

On the right side, there is an "Elements" panel with a grid of mathematical symbols and functions, including fractions, Greek letters, trigonometric functions, and mathematical constants. The bottom status bar shows the formula in a simplified, non-rendered format: `f(x,y) = {func e^(M) cdot sqrt(cos(x) + tan(y)) } over {M^(2)}`.

The screenshot shows the OpenOffice.org Math application window titled "Untitled1 - OpenOffice.org Math". The menu bar includes File, Edit, View, Format, Tools, Window, and Help. The toolbar contains various icons for mathematical operations and editing. The main workspace displays the gradient formula:

$$\text{grad } \varphi = \nabla \varphi = \frac{\partial \varphi}{\partial x} \vec{e}_x + \frac{\partial \varphi}{\partial y} \vec{e}_y + \frac{\partial \varphi}{\partial z} \vec{e}_z$$

Below the workspace, a scrollable area shows the LaTeX source code for the formula:

```
grad \varphi = nabla \varphi = {\{\partial \varphi\} \over {\partial x}} \widevec{e sub x} + {\{\partial \varphi\} \over {\partial y}} \widevec{e sub y} + {\{\partial \varphi\} \over {\partial z}} \widevec{e sub z}
```

The status bar at the bottom indicates a zoom level of 150%.



2.4. Collaboration (storage and sharing)



2.5. Contributing to an open source project *what does it mean?*

- Helping the Community
 - Answer questions on forums
 - Share extensions or plugins
- Write or improve documentation
- Report bugs
- Suggest or implement features
- Maintain projects
- Promote and share the software



2.5. Contributing to an open source project *what does it mean?*

- Helping the Community
- Write or improve documentation

- Write tutorials
- Report issues in official documentation
- Suggest improvements
- Translate documentation or the software

- Report bugs
- Suggest or implement features
- Maintain projects
- Promote and share the software



2.5. Contributing to an open source project *what does it mean?*

- Helping the Community
- Write or improve documentation
- Report bugs

- Suggest or implement features
- Maintain projects
- Promote and share the software

- Test new versions (especially beta releases)
- Report issues by opening tickets
- Help reproduce and understand problems
- Write fixes / patches



2.5. Contributing to an open source project *what does it mean?*

- Helping the Community
 - Write or improve documentation
 - Report bugs
 - Suggest or implement features
-
- Maintain projects
 - Promote and share the software

- Proposer de nouvelles fonctionnalités,
- Participer aux discussions,
- Implémenter des changements.



2.5. Contributing to an open source project *what does it mean?*

- Helping the Community
- Write or improve documentation
- Report bugs
- Suggest or implement features
- Maintain projects

- Help review code
- Assist with integrating contributions
- Support new version releases

- Promote and share the software



2.5. Contributing to an open source project *what does it mean?*

- Helping the Community
- Write or improve documentation
- Report bugs
- Suggest or implement features
- Maintain projects
- Promote and share the software

- Promote the software
- Create packages for different distributions



Questions to Ask Before Contributing

- Is the project free? (Which license?)
- Does it accept contributions?
- Is it active? (Date of last commit, last ticket, are tickets addressed?)



How to Contribute

- Start by understanding and respecting the rules (written and unwritten)
- Read the documentation: LICENSE, README, CONTRIBUTING, CODE_OF_CONDUCT, etc.
- Get familiar with the project: browse tickets, forums, chats, and follow project activity
- Generic rules:
 - Be polite
 - Be patient (maintainers owe nothing, not even attention)
 - Answer questions calmly
 - Check documentation before asking questions



How to Contribute: Bug Report

If you find a bug while using free software, report it to the software maintainers.

- 1. Locate the ticketing system / bug tracker.**
- 2. Search with different keywords to see if the bug has already been reported.**
 - If it has, you can add new information to help reproduce or debug it.
- 3. If no similar ticket exists, create a new one.**
 - Include all useful info to reproduce the bug: software version, how you obtained/compiled it, your environment (OS, browser...), precise steps or files to reproduce the issue (keep it as short as possible).



How to Contribute: Bug Report

- Bug reports can accumulate over time; sometimes essential info is missing.
- You can help by trying to reproduce already reported bugs:
 - **If successful:** explain the environment and software version, clarify or simplify reproduction steps.
 - **If unsuccessful:** it may mean the bug is fixed or environment-specific.
- Check the contribution guide to ensure this type of contribution is useful and accepted.